# RAG Assignment: Baseline vs. Re-Ranking

Kushal Mukherjee (2025201072)

19 October 2026

GitHub: `https://github.com/Kushal-2002/rag_Assignment`

## 1 Setup

I used the FIDE Laws of Chess PDF (25 pages) as the document. It has 14 Articles followed by appendices for Rapidplay, Blitz, notation, blind players and Chess960. The rules are numbered clause by clause (3.8, 9.2, A.4 and so on), so I made one chunk per clause and cut any clause longer than 150 words into smaller pieces. This gave 132 chunks.

| Component | What I used |
|---|---|
| Bi-encoder | all-MiniLM-L6-v2 (384-dim embeddings) |
| Vector store | FAISS IndexFlatIP with normalised vectors (so inner product = cosine) |
| Cross-encoder | cross-encoder/ms-marco-MiniLM-L-6-v2 |
| LLM | llama3 (8B) on Ollama, temperature 0 |

Both pipelines use the same prompt. It asks the LLM to answer only from the three chunks it is given, and to say "Not found in the context" if the answer is not there. I did this because otherwise llama3 could answer from its own memory and I would not be able to see when retrieval failed. Its memory is not reliable anyway: when I asked it about castling with no context, it said the king moves three squares, which is wrong (it is two).

## 2 Bi-Encoder vs. Cross-Encoder

A bi-encoder encodes the query and the chunk separately. Each one becomes a single vector, and the score is just the cosine similarity between the two vectors, $s(q, d) = \cos(E(q), E(d))$. Since the chunk vectors do not depend on the query, I could compute them once and store them in FAISS. At query time only the query has to be encoded. The problem is that the query and chunk words never see each other. The whole chunk is squeezed into one vector before the model knows what the question is, so small but important words like "not", "in rapidplay" or "for a blind player" do not change the score much.

A cross-encoder takes the query and the chunk together as one input (`[CLS] query [SEP] chunk [SEP]`). Self-attention runs over all the tokens at once, so every word in the query can attend to every word in the chunk, and the model outputs one relevance score. This is more accurate, but nothing can be stored in advance. Every (query, chunk) pair needs its own forward pass when the query comes in.

## 3 Rank Shifts and Answer Comparison

To find failure cases, I wrote around 160 test questions. For each one I read the document and noted down which chunk actually contains the answer. Then I checked where that chunk ended

up in the bi-encoder ranking and in the cross-encoder ranking. I counted a query as a failure case if the correct chunk was outside the baseline top 3 but inside the top 3 after re-ranking. I kept 30 of these questions in the notebook, and 10 of them meet this condition:

| Type | Query | Clause | Rank (bi → cross) |
|---|---|---|---|
| Document-specific | In rapidplay, after how many moves can you no longer complain about piece placement? | A.4 | 7 → 1 |
| Keyword trap | In blind chess, how is castling announced? | E.2 | 6 → 1 |
| Keyword trap | What symbol is used to mark a check? | C.13 | 5 → 1 |
| Document-specific | For a blind player, when is a piece considered touched? | E.2 | 4 → 1 |
| Exception | What happens if the starting position of the pieces was wrong in rapidplay? | A.4 | 7 → 2 |
| Negation | Can a player not move a piece again once he released it? | 4.6 | 6 → 2 |
| Exception | In Chess960, does the king always move when castling? | F.3 | 5 → 2 |
| Document-specific | In rapidplay, does the arbiter rule on touch-move on his own? | A.4 | 5 → 2 |
| Keyword trap | What names are used for the files in blind chess? | E.2 | 8 → 3 |
| Exception | Do the special touch rules for blind players also apply to the sighted player? | E.2 | 7 → 3 |

Most of these failures come from the way this document is written. A rule is given in the main Articles and then a different version of it appears again in an appendix (Rapidplay, blind players, Chess960). For example, for "In blind chess, how is castling announced?", the bi-encoder mostly matched the word "castling", and its top 3 had the normal castling rule (3.8) and the Chess960 castling chunks. The cross-encoder looked at the full question and moved the blind-chess appendix chunk (which says "Lange Rochade" / "Kurze Rochade") up to rank 1. For the negation question about a released piece, the bi-encoder ranked 4.5 (touched pieces) and 7.3 (displaced pieces) higher because they share the words "piece" and "move". Its top 3 did have the second half of clause 4.6, but not the part that actually says the piece cannot be moved again. The cross-encoder brought that part up to rank 2.

For the selected queries, I also compared the two LLM answers:

| Query | Answer 1 (baseline) | Answer 2 (re-ranked) |
|---|---|---|
| Rapidplay, piece placement | Not found | After three moves no claim can be made about piece placement, board orientation or clock setting (correct) |
| Blind chess, castling | Not found | "Lange Rochade" and "Kurze Rochade" (correct) |
| Blind player, touched piece | Not found | When it is taken out of the securing aperture (correct) |
| Released piece | Not found | It cannot be moved to another square on this move (correct) |
| Chess960 castling | Not found | Not found (chunk was at rank 2) |
| Rapidplay, touch-move | Not found | Not found (chunk was at rank 2) |

Because of the strict prompt, the baseline did not really make up rules. It just left the answer out and said "Not found", even though the right clause was sitting at rank 4 to 7. In four of the cases, re-ranking turned this into a correct answer. In the other two, the right chunk did get into the top 3 but llama3 still said "Not found", so better retrieval did not always give a better answer. I think this is partly because llama3 8B is a small model.

I also saw a case where the answer was actually wrong. For "How is a win on time claimed in rapidplay?", the correct chunk was not in the bi-encoder's top 10 at all, so the cross-encoder could not help. Answer 2 then described the Appendix D draw claim as if it were the way to claim a win, which is a misstated rule.

Re-ranking did not always help. In 3 of the 30 queries it pushed the correct chunk out of the top 3. For example, for "Can I go to the bathroom during my game?" the right chunk went from rank 1 to rank 4. The cross-encoder was trained on MS MARCO web search data, not on rule books, which may explain this. Overall, the bi-encoder was already good on this document whenever the question used the same words as the rulebook. Re-ranking helped mostly when the question was about a variant of a rule or was phrased differently.

## 4 Latency

I timed each stage with `time.perf_counter()` on my laptop (Apple Silicon CPU). The numbers below are averages over the 7 selected queries.

| Stage | Baseline (ms) | Re-ranking (ms) |
|---|---|---|
| Bi-encoder retrieval (top 10) | 43 | 43 |
| Cross-encoder re-ranking (10 pairs) | – | 84 |
| LLM answer (llama3) | 2253 | 3683 |
| Total | 2296 | 3810 |

Re-ranking adds about 84 ms per query, which is around 8 ms per pair. The retrieval step goes from 43 ms to 127 ms, so it roughly triples, but compared with the whole pipeline it is only about 2%. Almost all of the time (more than 95%) is spent in the LLM. The LLM time is higher for the re-ranking pipeline, but I do not think that is caused by re-ranking. The baseline often just replied "Not found in the context", which is short, while the re-ranked pipeline gave longer real answers. The timings also changed quite a bit between queries (retrieval was anywhere from 10 to 92 ms), which is why I used averages.

## 5 Why Not Use the Cross-Encoder on Everything?

The main reason is that a cross-encoder cannot pre-compute anything. With a bi-encoder, all the chunks are embedded once, offline, and at query time we only encode the query and search the index. Approximate indexes like HNSW make this search very fast even for millions of vectors. A cross-encoder needs the query as part of its input, so it has to process every chunk again for every new query.

This means the cost grows linearly with the number of chunks. From my timing, one pair takes about 8 ms. Scoring all 132 chunks of this document would already take about 1.1 seconds per query, more than 25 times the 43 ms that the bi-encoder takes. For a collection of 1 million passages it would be about $10^6 \times 8$ ms $\approx 2.2$ hours for one query, which is not usable.

That is why the two are combined. The bi-encoder quickly narrows everything down to a small set of candidates (10 in my case), and the cross-encoder is only used to order those few more carefully. The downside is that the cross-encoder can only re-order what the bi-encoder gives it. If the right chunk is not in the top $k$, as in my "win on time in rapidplay" example, re-ranking cannot fix it. So $k$ has to be chosen as a balance between recall and re-ranking time.

## 6 Conclusion

On the Laws of Chess, re-ranking fixed 10 of my 30 test queries where the bi-encoder had left the correct clause at rank 4 to 8, and in several cases it turned a "Not found" answer into a correct

one. It cost about 84 ms extra per query. Most of the improvement came from questions about rule variants in the appendices, where the cross-encoder picked up words like “rapidplay” or “blind” that the bi-encoder mostly ignored. It was not perfect, though. It made a few rankings worse, and it cannot recover a chunk that the bi-encoder did not retrieve in the first place.